

Rezeta Medical ERP — Reporte de Bugs e Ingeniería de Software

Sistema Evaluado	Rezeta Medical ERP (Monorepo Node/React/NestJS/Prisma)	Fecha del Reporte	20 de Mayo, 2026
Evaluator	QA de Software Médico Senior / Arquitecto de Soluciones	Cobertura de Pruebas	≥ 95% por archivo (Vitest y Jest)
Bases de Datos	PostgreSQL (Prisma ORM)	Gestor de Paquetes	pnpm (versión 10.33.0)

1. Resumen Técnico de la Aplicación

La arquitectura técnica de **Rezeta** es robusta, compuesta por un monorepo que gestiona el frontend en React 18 (Vite, TailwindCSS, Radix UI) y el backend en NestJS + Prisma ORM. Los esquemas de validación de datos se comparten mediante Zod en `packages/shared`.

Esta auditoría técnica evalúa el estado del código frente al rediseño híbrido de consulta/protocolo y las reglas estrictas de desarrollo del proyecto. Destaca la reciente campaña de refactorización y resolución de errores realizada entre el 8 y el 19 de mayo, la cual resolvió la totalidad de las deficiencias críticas identificadas en el ciclo estático inicial.

2. Informe de Corrección de Bugs e Inconsistencias (Ciclo Reciente)

Se documentan a continuación las vulnerabilidades y errores corregidos en el último sprint de ingeniería:

1. Hook de Sugerencias de Protocolos Simulado (Fake Hook)

CORREGIDO

Síntoma: El hook frontend de la pasarela de consulta ignoraba el ID del paciente y devolvía los protocolos generales ordenados por fecha de actualización, mostrando los mismos datos a todos los pacientes.

Resolución Técnica: Se renombró el hook a `useProtocolRecommendations` (en `use-protocol-recommendations.ts`). Se conectó correctamente al endpoint real del backend `/v1/patients/:patientId/protocol-recommendations`. Ahora recupera la matriz de recomendaciones personalizadas calculada por el repositorio de recomendaciones del backend (historial paciente/médico y pesos de frecuencia/recencia).

2. Creación No Atómica de Consultas y Adjunto de Protocolos

CORREGIDO

Síntoma: La creación de una consulta con protocolo ejecutaba dos llamadas API secuenciales no transaccionales (`POST /v1/consultations` y `POST /v1/consultations/:id/protocols`). Si la segunda fallaba, se creaba una consulta huérfana vacía.

Resolución Técnica: Se añadió el campo opcional `protocolId` al esquema `CreateConsultationSchema` (en `packages/shared/src/schemas/consultation.ts`). En el backend (`consultations.service.ts`), se encapsularon ambas inserciones en una transacción de base de datos prisma (`prisma.$transaction`). El frontend colapsó la secuencia a una sola llamada de mutación atómica.

3. Persistencia de Preferencia de Vista en LocalStorage

CORREGIDO

Síntoma: La preferencia de visualización de la consulta (Tradicional SOAP vs. Canvas de Protocolos) se guardaba en el `localStorage` del navegador, lo que impedía que la preferencia siguiera al médico al cambiar de dispositivo.

Resolución Técnica: Se agregó la columna `preferences JSONB` a la tabla `users` en la base de datos (con migración Prisma). Se expusieron los endpoints `GET /v1/users/me/preferences` y `PATCH /v1/users/me/preferences` . El hook frontend `use-consultation-view-mode.ts` ahora reconcilia el estado local con el backend, sincronizando las preferencias de forma transversal.

4. Componente de Consulta Monolítico (God Component de 1200+ Líneas)

CORREGIDO

Síntoma: `Consulta.tsx` albergaba toda la lógica de los formularios SOAP, componentes de constantes vitales, modales de firma, enmiendas, y las vistas de canvas de protocolos.

Resolución Técnica: Se dividió la página en subcomponentes modulares colocados en la misma carpeta: `SoapView.tsx`, `ConsultationSidebar.tsx`, `SaveBadge.tsx`, `VitalsSection.tsx` y `use-soap-state.ts`. El archivo base `Consulta.tsx` (ahora en inglés `Consultation/index.tsx`) se redujo a unas 300 líneas, delegando la presentación a contenedores puros y simplificando el mantenimiento.

5. Colisión de Nombres en Servicios de Recomendaciones y Sugerencias

CORREGIDO

Síntoma: El servicio para recomendar protocolos al paciente (para la Gate) y el servicio de retroalimentación de plantillas (del sistema de aprendizaje) tenían rutas y nombres homónimos ("suggestions"), induciendo a errores de programación.

Resolución Técnica: Se renombró el módulo de retroalimentación de plantillas a `protocol-improvements` y su ruta a `/v1/protocols/:id/improvements`. El módulo de sugerencias de paciente se consolidó formalmente como `protocol-recommendations` bajo la ruta `/v1/patients/:id/protocol-recommendations`, eliminando cualquier ambigüedad semántica.

6. Duplicación de Campo Legacy en el Esquema de Consulta

CORREGIDO

Síntoma: Coexistían el campo legacy `protocolsApplied String[]` en la tabla `Consultation` y la relación uno a muchos con `ProtocolUsage`, duplicando el origen de la verdad sobre los protocolos utilizados.

Resolución Técnica: Se eliminó por completo el campo `protocolsApplied` del modelo Prisma y se creó una migración SQL (`drop_protocols_applied`) que eliminó físicamente la columna de la base de datos de forma segura, consolidando a `ProtocolUsage` como la única fuente de verdad.

3. Recomendaciones Técnicas Adicionales y Deuda Técnica Activa

A pesar de la sobresaliente corrección de bugs, quedan pendientes las siguientes acciones preventivas y mejoras de arquitectura:

1. Eliminación Definitiva de Columnas Obsoletas de Compatibilidad (`checkedState`)

El modelo `ProtocolUsage` aún almacena el estado de pasos completados de dos formas: a través del campo legacy `checkedState Json` (un mapa simple de booleanos) y mediante `modifications Json` (un registro histórico detallado de eventos de edición del médico).

Recomendación de Deuda Técnica: Actualmente, el frontend utiliza el helper `getCheckedStateFromModifications()` para derivar el estado desde el historial de modificaciones, pero la columna obsoleta `checkedState` sigue en la base de datos. Se debe programar una migración para removerla físicamente del esquema Prisma, evitando que futuros desarrolladores o módulos de análisis consuman el mapa de booleanos desactualizado.

2. Enmascaramiento de Errores e Integridad de Logs en Producción

Se ha integrado correctamente un filtro global de excepciones (`http-exception.filter.ts`) que desenmascara detalles de errores de infraestructura (como fallos en base de datos Prisma) solo en entornos de desarrollo (`NODE_ENV !== 'production'`) y registra de forma limpia los códigos de error en la tabla `AuditLog` .

- Se debe auditar periódicamente el tamaño de la tabla `AuditLog` . Al registrar mutaciones fallidas y metadatos JSON completos, se recomienda establecer una política de retención o particionado para evitar degradación de rendimiento a largo plazo en Postgres.

3. Cobertura de Pruebas Unitarias al Agregar Nuevos Componentes

El monorepo cuenta con una regla estricta de cobertura del **95% por archivo** en todas sus directrices.

- Cada vez que se introduzca un componente visual en `apps/web/src/components/ui/` o páginas del ERP, se debe emparejar obligatoriamente con su respectivo archivo `__tests__/*.test.tsx` que pruebe los estados de carga, error e interacción de usuario.
- Se debe evitar añadir excepciones al archivo `vitest.config.ts` para mantener la integridad de la base de código.

4. Documentación de Endpoints y Contratos OpenAPI/Swagger

Para garantizar que las herramientas de prueba automatizadas y socios de integración externos puedan consumir la API sin fricciones, todos los controladores de NestJS en `apps/api/src/modules/` deben estar completamente decorados con los decoradores de `@nestjs/swagger` (tales como `@ApiTags()` , `@ApiOperation()` , `@ApiResponse()`).